

Einsendeaufgabe TST-E1 – Entwicklungsdokumentation

Modul: Softwaretechnik (SWT)

Aufgabe: TST-E1 – Projektaufgabe Testen

Student: Olaf Gustke

Einleitung

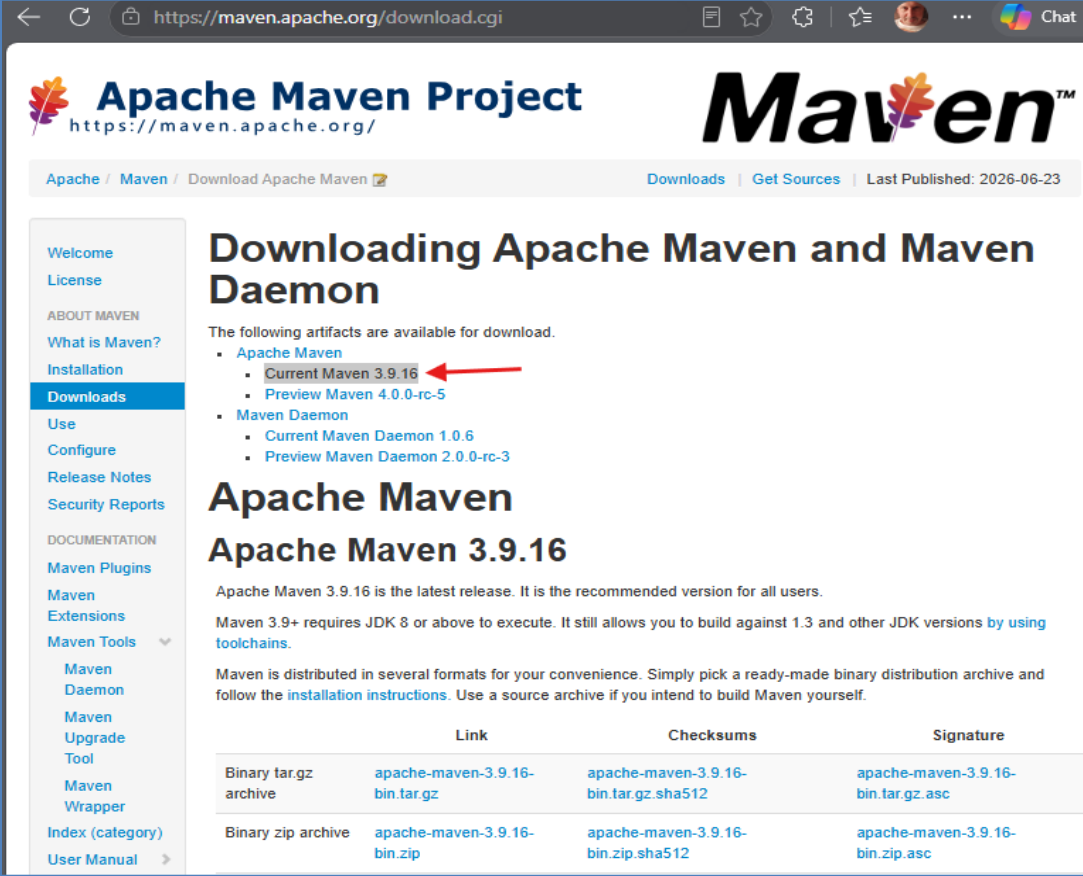
Im Rahmen dieser Einsendeaufgabe wurde eine Java-Anwendung nach den im Modul vermittelten Testverfahren entwickelt. Ziel war nicht nur die Erstellung lauffähigen Codes, sondern insbesondere die nachvollziehbare Dokumentation des Entwicklungsprozesses einschließlich auftretender Probleme, deren Lösungen sowie des Einsatzes von KI-Unterstützung.

1. Vorbereitung der Entwicklungsumgebung

Zu Beginn wurde entschieden, die Aufgabe in Java umzusetzen. Obwohl Python aufgrund der einfachen Unterstützung von Mocking ebenfalls möglich gewesen wäre, fiel die Wahl auf Java, da bereits Erfahrungen aus vorherigen Modulen vorhanden waren.

Anschließend wurden die notwendigen Werkzeuge installiert und eingerichtet:

- Java Development Kit (JDK)
- Visual Studio Code
- Apache Maven
- Git



Apache Maven Project
<https://maven.apache.org/>

Apache / Maven / Download Apache Maven

Downloading Apache Maven and Maven Daemon

The following artifacts are available for download.

- Apache Maven
 - Current Maven 3.9.16** (highlighted with a red arrow)
 - Preview Maven 4.0.0-rc-5
- Maven Daemon
 - Current Maven Daemon 1.0.6
 - Preview Maven Daemon 2.0.0-rc-3

Apache Maven

Apache Maven 3.9.16

Apache Maven 3.9.16 is the latest release. It is the recommended version for all users.

Maven 3.9+ requires JDK 8 or above to execute. It still allows you to build against 1.3 and other JDK versions [by using toolchains](#).

Maven is distributed in several formats for your convenience. Simply pick a ready-made binary distribution archive and follow the [installation instructions](#). Use a source archive if you intend to build Maven yourself.

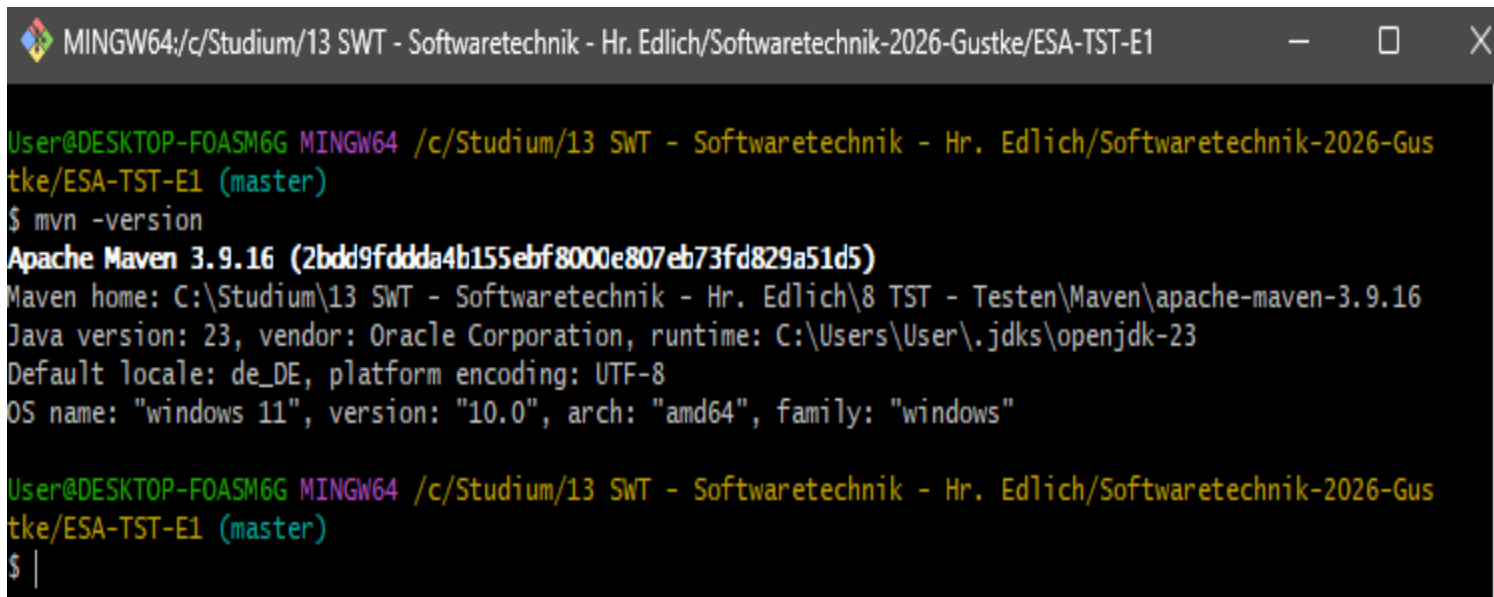
	Link	Checksums	Signature
Binary tar.gz archive	apache-maven-3.9.16-bin.tar.gz	apache-maven-3.9.16-bin.tar.gz.sha512	apache-maven-3.9.16-bin.tar.gz.asc
Binary zip archive	apache-maven-3.9.16-bin.zip	apache-maven-3.9.16-bin.zip.sha512	apache-maven-3.9.16-bin.zip.asc

Abbildung 1: Download von Apache Maven.

2. Installation von Apache Maven

Apache Maven wurde von der offiziellen Projektseite heruntergeladen und lokal installiert.

Während der Einrichtung stellte sich heraus, dass Maven zunächst nicht über die Kommandozeile erreichbar war. Ursache war, dass sich der Maven-Ordner noch nicht in der Windows-PATH-Umgebungsvariable befand. Nach dem Eintragen des Ordners ...\\apache-maven-3.9.16\\bin konnte Maven erfolgreich verwendet werden.



```
MINGW64:/c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-TST-E1
User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-TST-E1 (master)
$ mvn -version
Apache Maven 3.9.16 (2bdd9fddda4b155ebf8000e807eb73fd829a51d5)
Maven home: C:\Studium\13 SWT - Softwaretechnik - Hr. Edlich\8 TST - Testen\Maven\apache-maven-3.9.16
Java version: 23, vendor: Oracle Corporation, runtime: C:\Users\User\.jdk\openjdk-23
Default locale: de_DE, platform encoding: UTF-8
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"
User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-TST-E1 (master)
$ |
```

Abbildung 2: Überprüfung der erfolgreichen Maven-Installation mit mvn -version.

Nach der erfolgreichen Installation wurde das Maven-Projekt in Visual Studio Code geöffnet. Dabei wurde geprüft, ob die Projektdatei pom.xml vorhanden ist und das Projekt für die weitere Entwicklung vorbereitet werden konnte.

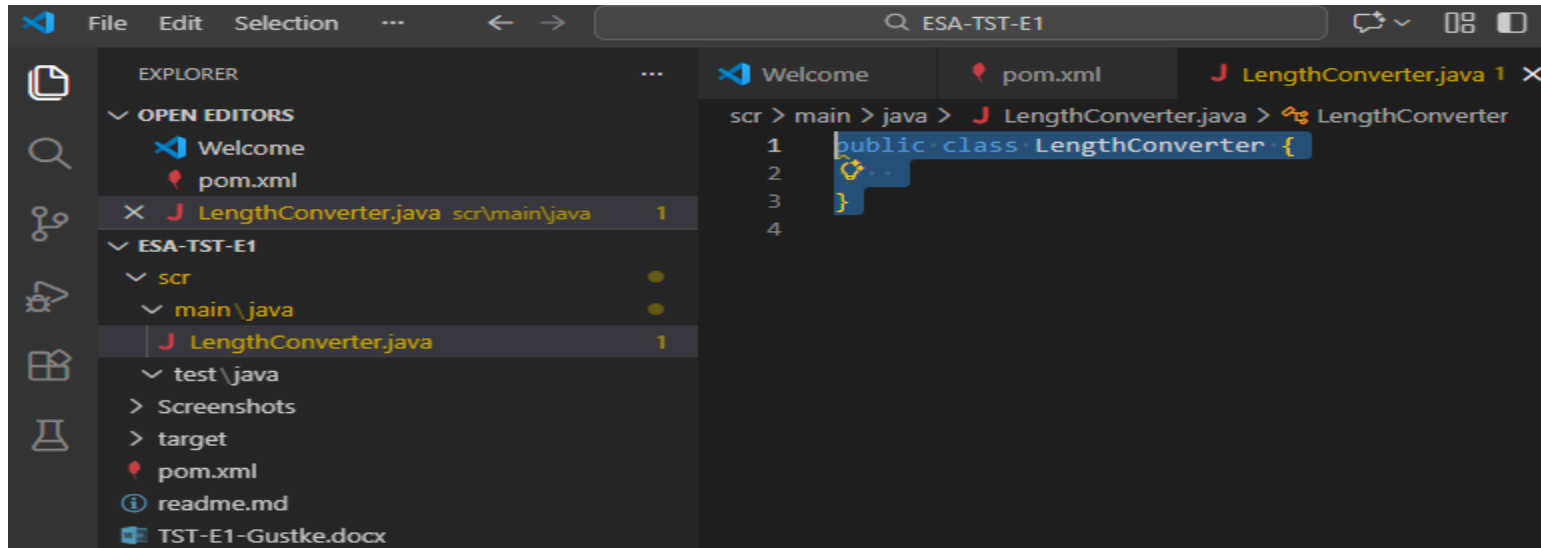


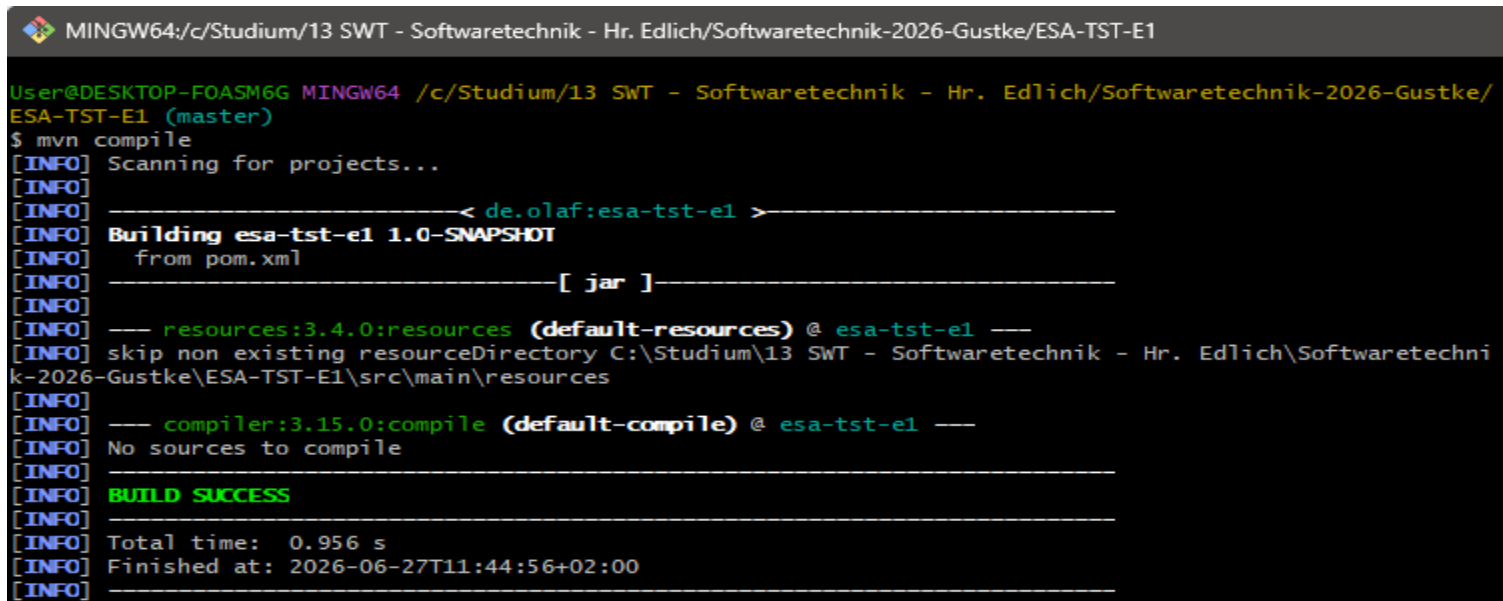
Abbildung 3: Geöffnetes Maven-Projekt ESA-TST-E1 in Visual Studio Code.

Bei der ersten Verwendung von Maven trat zunächst eine Fehlermeldung auf. Der Befehl `mvn compile` wurde versehentlich im Repository-Hauptverzeichnis ausgeführt. Dadurch konnte Maven die Datei `pom.xml` nicht finden. Nach der Analyse der Fehlermeldung wurde erkannt, dass zunächst in den Projektordner `ESA-TST-E1` gewechselt werden musste. Anschließend konnte Maven das Projekt korrekt erkennen und ausführen.

```
The goal you specified requires a project to execute but there is no POM in  
this directory...
```

Abbildung 4: Fehlermeldung durch Ausführung von `mvn compile` im falschen Verzeichnis.

Nach dem Wechsel in das richtige Projektverzeichnis konnte Maven das Projekt erfolgreich kompilieren. Dabei wurden automatisch die in der Datei pom.xml definierten Abhängigkeiten heruntergeladen und eingebunden. Der erfolgreiche Build bestätigte, dass die Entwicklungsumgebung korrekt eingerichtet war und mit der eigentlichen Programmierung begonnen werden konnte.

A screenshot of a terminal window with a dark background and light-colored text. The window title bar at the top reads "MINGW64:/c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-TST-E1". The terminal shows a command prompt where the user has entered "\$ mvn compile". The output consists of several lines of status messages: "[INFO] Scanning for projects...", "[INFO] Building esa-tst-e1 1.0-SNAPSHOT from pom.xml", "[INFO] [jar]", "[INFO] --- resources:3.4.0:resources (default-resources) @ esa-tst-e1 ---", "[INFO] skip non existing resourceDirectory C:\Stadium\13 SWT - Softwaretechnik - Hr. Edlich\Softwaretechnik-2026-Gustke\ESA-TST-E1\src\main\resources", "[INFO] --- compiler:3.15.0:compile (default-compile) @ esa-tst-e1 ---", "[INFO] No sources to compile", "[INFO] BUILD SUCCESS", "[INFO] Total time: 0.956 s", "[INFO] Finished at: 2026-06-27T11:44:56+02:00".

```
MINGW64:/c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-TST-E1
User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-TST-E1 (master)
$ mvn compile
[INFO] Scanning for projects...
[INFO]
[INFO] -----< de.olaf:esa-tst-e1 >-----
[INFO] Building esa-tst-e1 1.0-SNAPSHOT
[INFO]    from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ esa-tst-e1 ---
[INFO] skip non existing resourceDirectory C:\Stadium\13 SWT - Softwaretechnik - Hr. Edlich\Softwaretechnik-2026-Gustke\ESA-TST-E1\src\main\resources
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ esa-tst-e1 ---
[INFO] No sources to compile
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 0.956 s
[INFO] Finished at: 2026-06-27T11:44:56+02:00
[INFO]
```

Abbildung 5: Erfolgreicher Build des Maven-Projekts mit mvn compile.

3. Erste Schritte mit Maven

Zu Beginn der Bearbeitung war zunächst unklar, welche Aufgaben Maven innerhalb eines Java-Projekts übernimmt. Im Verlauf der Einrichtung wurde deutlich, dass Maven nicht nur das Kompilieren des Quellcodes ermöglicht, sondern auch externe Bibliotheken verwaltet, Unit-Tests ausführt und einen standardisierten Projektaufbau unterstützt. Dieses Verständnis erleichterte die weitere Bearbeitung erheblich. Während der Einrichtung wurde deutlich, dass Maven mehrere Aufgaben übernimmt:

- Verwaltung externer Bibliotheken
 - Kompilieren des Projektes
 - Ausführen von Tests
 - Einheitlicher Projektaufbau
-

4. Erstellung der ersten Java-Klasse

Nach der erfolgreichen Einrichtung der Entwicklungsumgebung begann die eigentliche Umsetzung der Einsendeaufgabe. Als Beispielanwendung wurde ein Längenumrechner (Inch ↔ Zentimeter) gewählt. Eine solche Umrechnung kann beispielsweise bei Reisen ins Ausland hilfreich sein, etwa um die in Zoll angegebenen Kofferabmessungen von Fluggesellschaften schnell in Zentimeter umzurechnen.

Mit der Einrichtung des Projekts wurde die erste Java-Klasse `LengthConverter` angelegt. Diese ist bereits in Abbildung 3 zu erkennen und bildet die Grundlage für die nachfolgenden Unit-Tests sowie die Entwicklung nach dem Test-Driven-Development-(TDD)-Prinzip.

5. Test-Driven Development (TDD)

Nach der Einrichtung des Projekts begann die eigentliche testgetriebene Entwicklung nach dem Test-Driven-Development-(TDD)-Prinzip. Dabei wird zunächst ein Test erstellt, der aufgrund der noch fehlenden Implementierung fehlschlägt („Red“). Anschließend wird der notwendige Programmcode entwickelt, bis der Test erfolgreich durchläuft („Green“). Abschließend kann der Code bei Bedarf verbessert werden („Refactor“).

5.1 Erster Unit-Test und TDD-Schritt „Red“

Während des ersten Testlaufs zeigte Maven zunächst an, dass keine Java-Quelldateien gefunden wurden. Ursache war eine fehlerhafte Projektstruktur, die anschließend korrigiert wurde.

Beim ersten Testlauf trat wie erwartet ein Fehler auf. Maven konnte die Methode `inchToCentimeter(int)` nicht finden, da diese in der Klasse `LengthConverter` noch nicht implementiert war. Dieser Fehler zeigt den ersten TDD-Schritt „Red“: Der Test beschreibt bereits die gewünschte Funktion, der produktive Code erfüllt sie aber noch nicht.

```
MINGW64:/c:/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-TST-E1

User@DESKTOP-FOASM6G MINGW64 /c:/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-TST-E1 (master)
$ mvn test
[INFO] Scanning for projects...
[INFO]
[INFO] -----< de.olaf:esa-tst-e1 >-----
[INFO] Building esa-tst-e1 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ esa-tst-e1 ---
[INFO] skip non existing resourceDirectory C:\Studium\13 SWT - Softwaretechnik - Hr. Edlich\Softwaretechnik-2026-Gustke\ESA-TST-E1\src\
main\resources
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ esa-tst-e1 ---
[INFO] Recompiling the module because of added or removed source files.
[INFO] Compiling 1 source file with javac [debug target 17] to target\classes
[WARNING] Systemmodulpfad ist nicht zusammen mit -source 17 festgelegt
Wenn Sie den Speicherort der Systemmodule nicht festlegen, kann dies zu Klassendateien führen, die auf JDK 17 nicht ausgeführt werden
können
--release 17 wird anstelle von -source 17 -target 17 empfohlen, weil dadurch der Speicherort der Systemmodule automatisch festgeleg
t wird
[INFO]
[INFO] --- resources:3.4.0:testResources (default-testResources) @ esa-tst-e1 ---
[INFO] skip non existing resourceDirectory C:\Studium\13 SWT - Softwaretechnik - Hr. Edlich\Softwaretechnik-2026-Gustke\ESA-TST-E1\src\
test\resources
[INFO]
[INFO] --- compiler:3.15.0:testCompile (default-testCompile) @ esa-tst-e1 ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 1 source file with javac [debug target 17] to target\test-classes
[INFO] -----
[WARNING] COMPILATION WARNING :
[INFO] -----
[WARNING] Systemmodulpfad ist nicht zusammen mit -source 17 festgelegt
Wenn Sie den Speicherort der Systemmodule nicht festlegen, kann dies zu Klassendateien führen, die auf JDK 17 nicht ausgeführt werden
können
--release 17 wird anstelle von -source 17 -target 17 empfohlen, weil dadurch der Speicherort der Systemmodule automatisch festgeleg
t wird
```

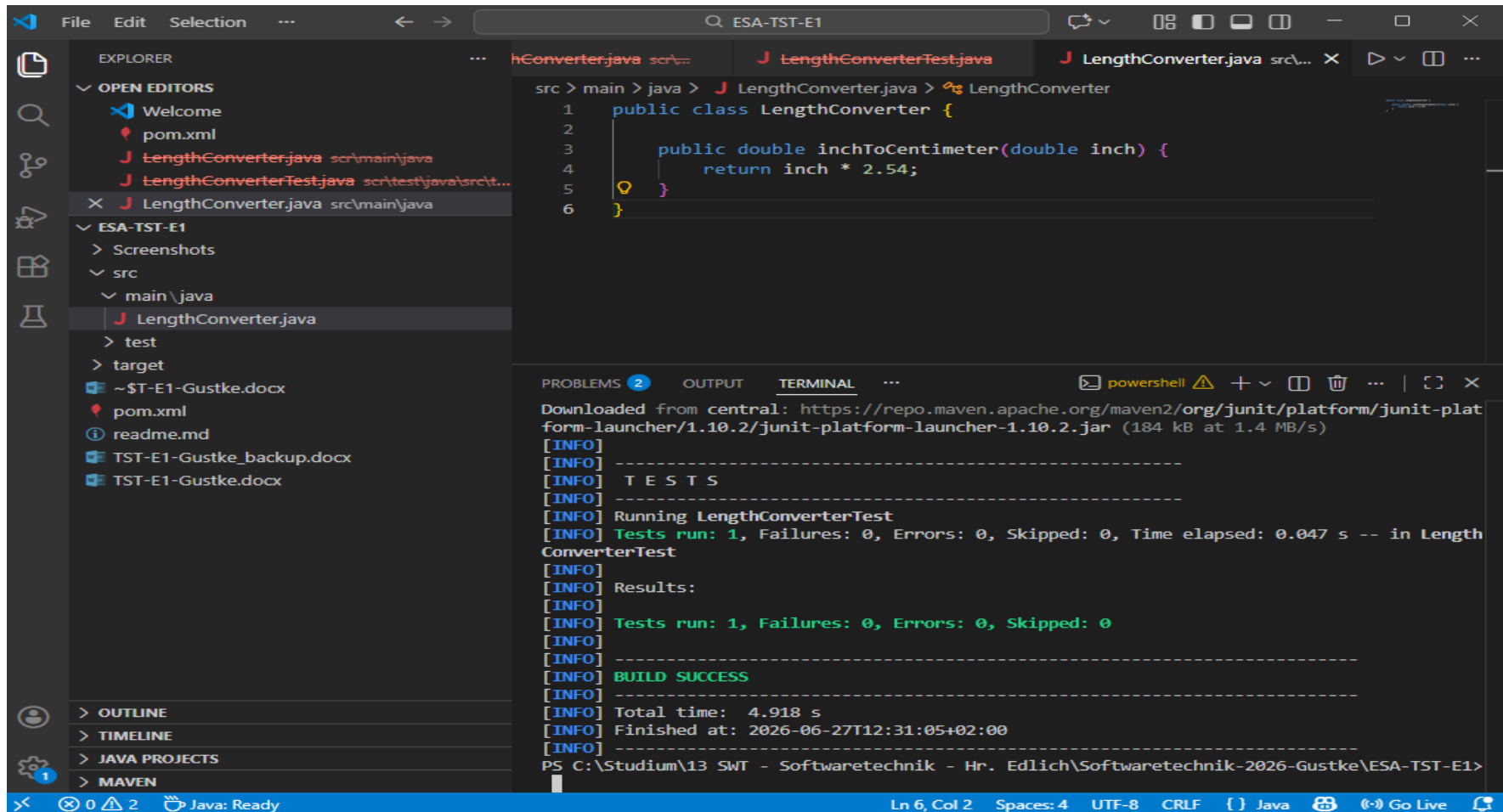


```
[INFO] -----
[INFO] -----
[ERROR] COMPILATION ERROR :
[INFO] -----
[ERROR] /C:/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-TST-E1/src/test/java/LengthConverterTest.java
:[10,34] Symbol nicht gefunden
    Symbol: Methode inchToCentimeter(int)
    Ort: Variable converter von Typ LengthConverter
[INFO] 1 error
[INFO] -----
[INFO] BUILD FAILURE
[INFO] -----
[INFO] Total time: 1.421 s
[INFO] Finished at: 2026-06-27T12:18:51+02:00
[INFO] -----
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.15.0:testCompile (default-testCompile) on project esa-t
st-e1: Compilation failure
[ERROR] /C:/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-TST-E1/src/test/java/LengthConverterTest.java
:[10,34] Symbol nicht gefunden
[ERROR]     Symbol: Methode inchToCentimeter(int)
[ERROR]     Ort: Variable converter von Typ LengthConverter
[ERROR] -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException

User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-TST-E1 (master)
$ |
```

Abbildung 6: Fehlgeschlagener Testlauf im TDD-Schritt „Red“ Teil 1 & Teil 2

Nach der Fehlermeldung wurde die Methode `inchToCentimeter(double inch)` in der Klasse `LengthConverter` implementiert. Anschließend wurde der Test erneut ausgeführt. Dieses Mal konnte die Methode erfolgreich kompiliert werden und der Unit-Test wurde ohne Fehler bestanden. Damit war der zweite Schritt des TDD-Prinzips („Green“) erreicht. Die implementierte Funktion erfüllt nun die zuvor definierte Anforderung.



```
src > main > java > J LengthConverter.java > LengthConverter
1 public class LengthConverter {
2
3     public double inchToCentimeter(double inch) {
4         return inch * 2.54;
5     }
6 }
```

```
Downloaded from central: https://repo.maven.apache.org/maven2/org/junit/platform/junit-platform-launcher/1.10.2/junit-platform-launcher-1.10.2.jar (184 kB at 1.4 MB/s)
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running LengthConverterTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.047 s -- in LengthConverterTest
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.918 s
[INFO] Finished at: 2026-06-27T12:31:05+02:00
[INFO] -----
PS C:\Studium\13 SWT - Softwaretechnik - Hr. Edlich\Softwaretechnik-2026-Gustke\ESA-TST-E1>
```

Abbildung 7: Erfolgreicher Unit-Test nach Implementierung der Methode `inchToCentimeter()` (TDD-Schritt „Green“).

5.2 Prüfung einer Exception

Neben der korrekten Umrechnung wurde zusätzlich überprüft, wie sich die Anwendung bei einer ungültigen Eingabe verhält. Da negative Längen in diesem Anwendungsfall nicht sinnvoll sind, wurde die Methode `inchToCentimeter()` so erweitert, dass sie bei einem negativen Eingabewert eine `IllegalArgumentException` auslöst.

Zur Überprüfung wurde ein weiterer Unit-Test implementiert. Der Test verwendet `assertThrows()`, um sicherzustellen, dass bei einer negativen Eingabe die erwartete Exception ausgelöst wird. Der erfolgreiche Testlauf bestätigt, dass sowohl die Umrechnung als auch die Behandlung fehlerhafter Eingaben korrekt funktionieren. Damit sind die Anforderungen der Aufgabe A1 vollständig erfüllt.

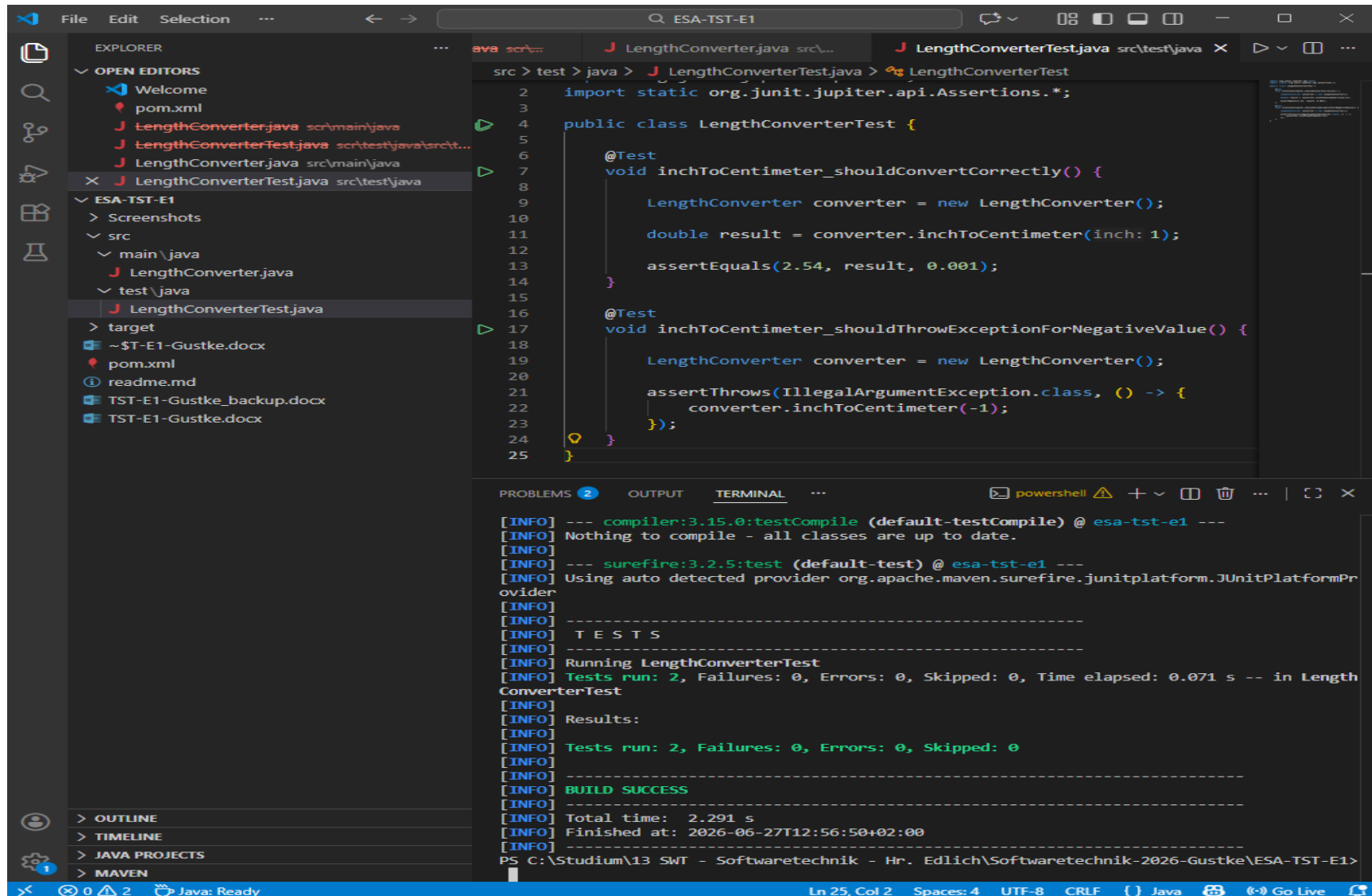


Abbildung 8: Erfolgreicher Testlauf der Unit-Tests einschließlich Exception-Test.

A2 – Shopping Cart im TDD-Modus.

5.3 Beginn der Entwicklung des Shopping Carts

Nach Abschluss der Unit-Tests wurde mit der Umsetzung der zweiten Aufgabenstellung begonnen. Hierfür wurde zunächst die Klasse ShoppingCart als Grundlage für die weitere Entwicklung angelegt. Die eigentliche Implementierung erfolgt anschließend schrittweise nach dem TDD-Prinzip, sodass jede neue Funktion zunächst durch einen Test beschrieben und anschließend umgesetzt wird.

Während der Einrichtung der Teststruktur wurde versehentlich eine verschachtelte Ordnerstruktur (`src/test/java/src/test/java`) erzeugt. Ursache war die versehentliche Angabe eines vollständigen Pfades beim Erstellen einer neuen Datei in VS Code. Nach der Korrektur entsprach die Projektstruktur wieder der von Maven erwarteten Standardstruktur.

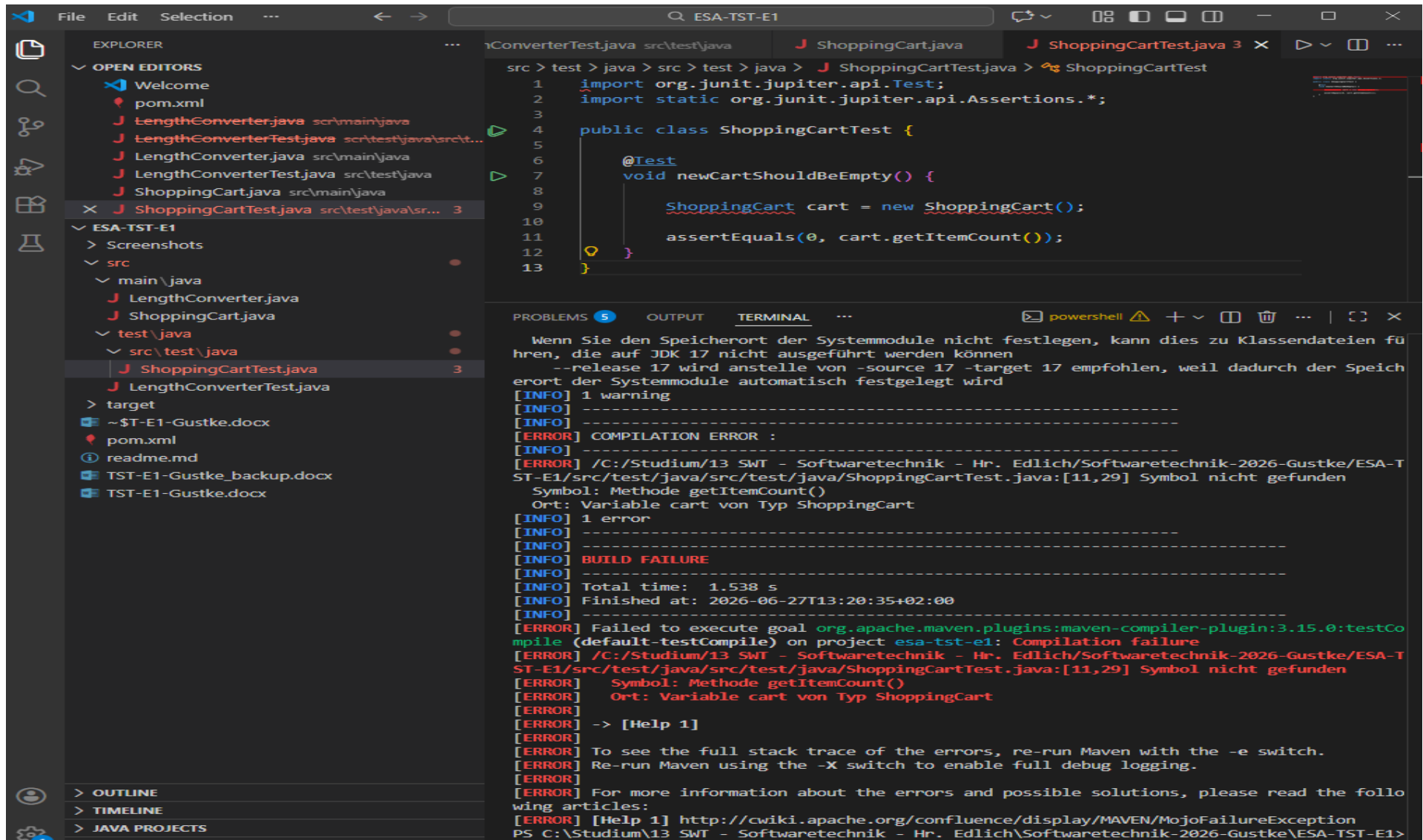


Abbildung 9: Fehlgeschlagener Testlauf des Shopping Carts im TDD-Schritt „Red“ vor der Implementierung der Methode getItemCount().

Nach dem fehlgeschlagenen Test wurde die Methode `getItemCount()` in der Klasse `ShoppingCart` implementiert. Die Methode liefert zunächst den Wert 0 zurück, da sich zu diesem Zeitpunkt noch kein Artikel im Warenkorb befindet. Der anschließende Testlauf verlief erfolgreich. Damit wurde der Green-Schritt des TDD-Prinzips erreicht und die erste Funktionalität des Shopping Carts erfolgreich umgesetzt.

Anschließend wurde die Testklasse um einen weiteren Unit-Test erweitert, der überprüft, ob beim Erzeugen eines neuen Shopping Carts ein gültiges Objekt erstellt wird.

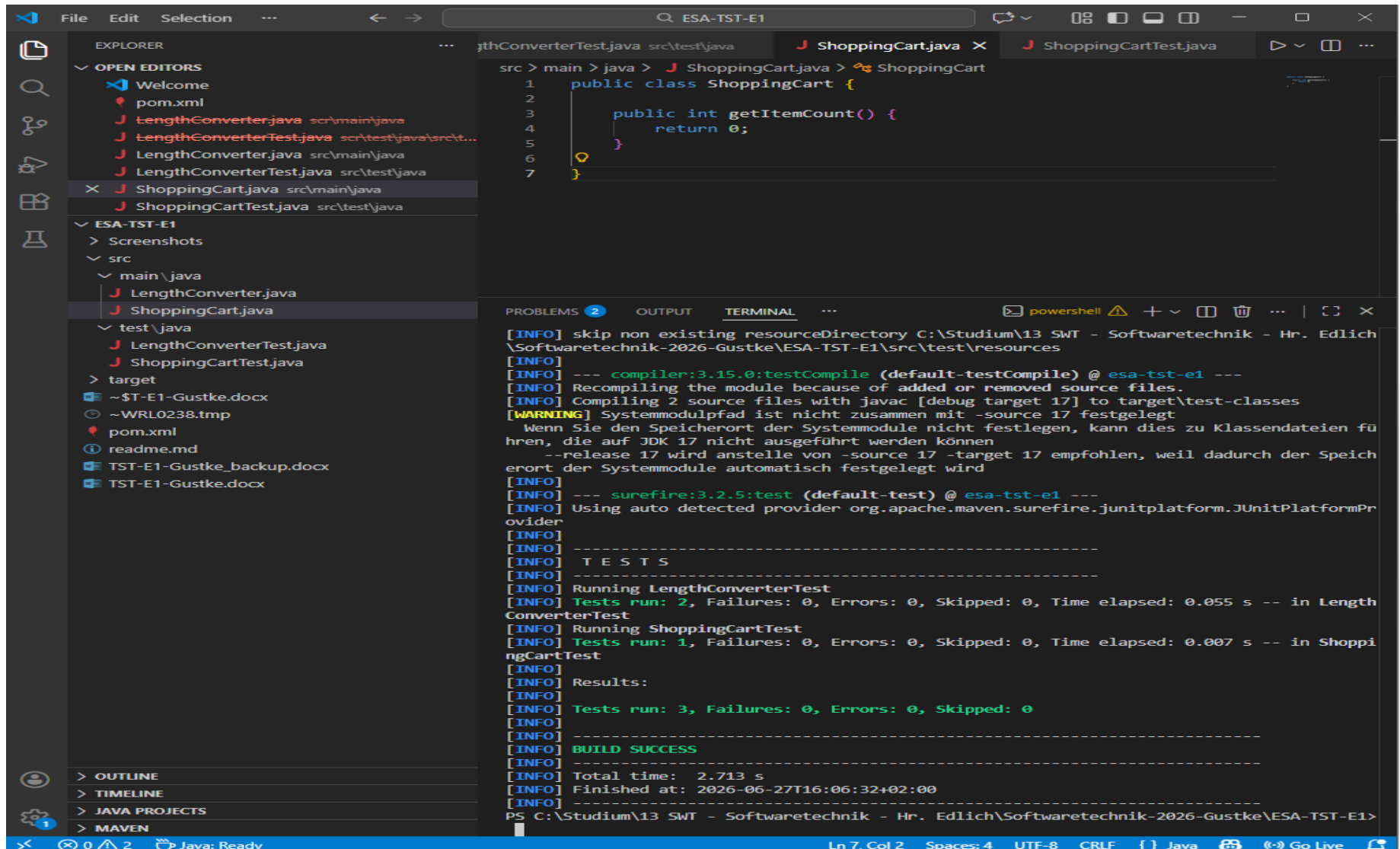


Abbildung 10: Erfolgreicher Testlauf nach Implementierung der Methode getItemCount() im Shopping Cart (TDD-Schritt „Green“).

Der anschließende Testlauf zeigte, dass beide Unit-Tests erfolgreich ausgeführt wurden. Neben der Prüfung der Artikelanzahl wurde zusätzlich überprüft, ob beim Erzeugen eines neuen Shopping Carts ein gültiges Objekt erstellt wird. Damit befindet sich der Shopping Cart nach seiner Initialisierung in einem definierten Ausgangszustand.

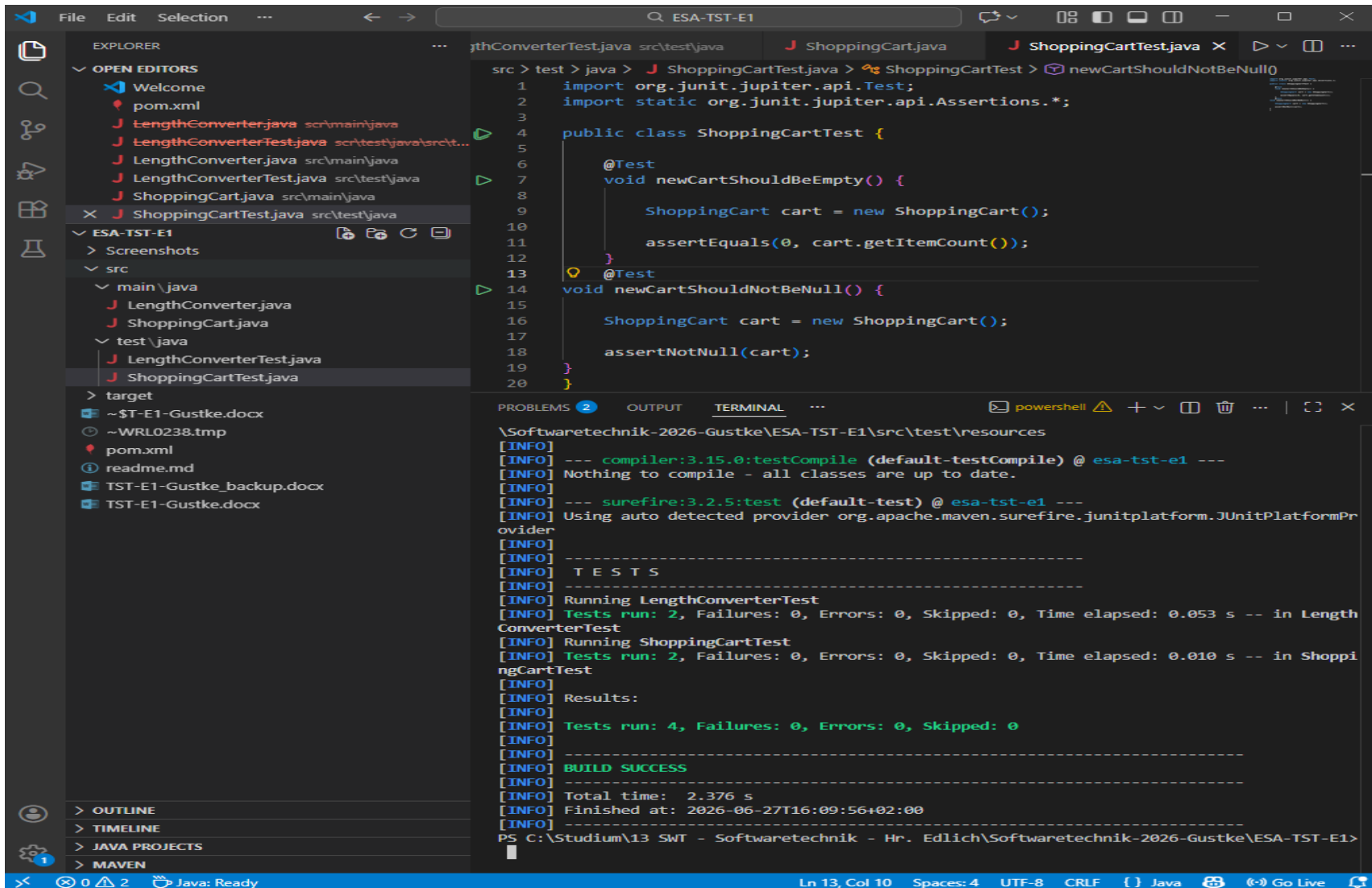


Abbildung 11: Erfolgreicher Testlauf des Shopping Carts mit zwei Unit-Tests (BUILD SUCCESS).

5.4 Zusammenfassung der TDD-Entwicklung

Die Entwicklung erfolgte konsequent nach dem Test-Driven-Development-(TDD)-Prinzip. Zunächst wurden die erwarteten Funktionen in Form von Unit-Tests beschrieben. Anschließend wurde jeweils nur der notwendige Programmcode implementiert, bis die Tests erfolgreich ausgeführt werden konnten. Dieses Vorgehen erleichterte die schrittweise Entwicklung, machte Fehler frühzeitig sichtbar und sorgte für eine nachvollziehbare Dokumentation des Entwicklungsprozesses.

6. Fazit

Im Rahmen der Einsendeaufgabe wurden die Grundlagen des Test-Driven Development mit Maven und JUnit praktisch umgesetzt. Nach der erfolgreichen Einrichtung der Entwicklungsumgebung wurden zunächst Unit-Tests für einen Längenumrechner und anschließend für einen einfachen Shopping Cart entwickelt. Während der Bearbeitung traten verschiedene Probleme auf, beispielsweise bei der Maven-Konfiguration und der Projektstruktur. Diese konnten systematisch analysiert und behoben werden. Insgesamt entstand eine funktionsfähige Lösung, deren Entwicklungsschritte durch Testläufe und Screenshots nachvollziehbar dokumentiert wurden.